Assignment 01: Selenium Webdriver

ENSF 519 L02 - Group 9

Team Members

Muhammad Zain | UCID: 30162484

Fateh Ali Syed Bukhari | UCID: 30123431

Application under test: RCC - Resume & Career Coach
Local stack at http://localhost:3000, driven with Python and Selenium WebDriver (Chrome)

Overview

This document describes 34 Selenium WebDriver test cases written in Python for the RCC web application. The suite is split into three groups: 18 scripts that run as a logged-out visitor, 1 script that completes a full signup with email verification, and 15 scripts that run against a signed-in account. Each script opens a headed Chrome window, performs the actions listed, and asserts the expected result using explicit waits.

All tests target the local development stack (React on port 3000, Flask on port 5000, PostgreSQL in Docker). Shared helpers live in `selenium_tests/support/`: `browser.py` creates the driver and opens pages with a cleared session, `auth.py` signs in as the permanent test account, `pool.py` manages the 100 allow-listed `.test` addresses used for signup, `form.py` sets React-controlled fields, and `bootstrap.py` creates the permanent account once per database.

Running a script: activate the virtual environment in `selenium_tests/` and run `python <folder>\<script>.py`. A script that returns to the prompt without a traceback has passed.

Test Case Index

#	Test case	Script	Group
1	Home page loads	`test_home_loads.py`	Logged out
2	Home navigation links scroll to sections	`test_home_nav_sections.py`	Logged out
3	FAQ accordion expands	`test_faq_expand.py`	Logged out
4	Public theme toggle	`test_theme_public.py`	Logged out
5	Nav Sign in link opens login	`test_nav_sign_in.py`	Logged out
6	Hero CTA opens signup	`test_cta_signup.py`	Logged out
7	Open dashboard redirects guests	`test_open_dashboard_logged_out.py`	Logged out
8	Terms page loads	`test_terms_page.py`	Logged out
9	Footer Terms link	`test_footer_terms.py`	Logged out
10	Footer Contact popover	`test_footer_contact.py`	Logged out
11	Signup form fields	`test_signup_form.py`	Logged out
12	Auth tab switching	`test_auth_tab_switch.py`	Logged out
13	Signup rejects a short password	`test_signup_short_password.py`	Logged out
14	Signup rejects an unlisted test address	`test_signup_rejects_unlisted.py`	Logged out
15	Login form fields	`test_login_form.py`	Logged out
16	Login with wrong credentials	`test_login_bad_password.py`	Logged out
17	Protected routes redirect to login	`test_protected_redirects.py`	Logged out
18	Legacy routes redirect to login	`test_legacy_redirects.py`	Logged out
19	Full signup and email verification	`test_signup_full.py`	Signup
20	Login succeeds	`test_login_success.py`	Logged in
21	Overview page content	`test_overview_content.py`	Logged in
22	Sidebar navigation	`test_sidebar_nav.py`	Logged in

#	Test case	Script	Group
23	New analysis button	`test_new_analysis_button.py`	Logged in
24	Analyze page fields	`test_analyze_page.py`	Logged in
25	Attach a resume file	`test_analyze_attach_file.py`	Logged in
26	Run a full analysis	`test_analyze_run.py`	Logged in
27	Interview page fields	`test_interview_page.py`	Logged in
28	Job description enables Begin interview	`test_interview_jd_enables.py`	Logged in
29	History page	`test_history_page.py`	Logged in
30	Settings profile	`test_settings_profile.py`	Logged in
31	Delete account dialog	`test_settings_delete_modal.py`	Logged in
32	Dashboard theme toggle	`test_dashboard_theme.py`	Logged in
33	Sidebar collapse and expand	`test_sidebar_collapse.py`	Logged in
34	Logout	`test_logout.py`	Logged in

Part 1 - Logged-out Test Cases

Test Case 1: Home page loads

`selenium_tests/logged_out_scripts/test_home_loads.py`

Description

Checks that the public landing page of RCC renders correctly for a visitor who is not signed in. It confirms the brand heading, the hero tagline, and the two primary call-to-action buttons are all present and visible.

Steps

1 Open Chrome and navigate to / with cookies and localStorage cleared.
2 Wait for the h1 heading to contain the text RCC.
3 Wait for the hero tagline paragraph to contain Build smarter resumes.
4 Wait for the Get started free button to be visible and clickable.
5 Wait for the Open dashboard button to be visible and clickable.

Expected Result

The heading reads RCC, the tagline contains Build smarter resumes., and both Get started free and Open dashboard are visible on the page.

Python Script (Selenium WebDriver)

```
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.browser import make_driver, open_fresh, wait

def test_home_loads():
    driver = make_driver()

    try:
        # Open the home page
        open_fresh(driver, "/")

        # Wait for the homepage to load (if required) - h1 is RCC. Tagline contains Build smarter
            resumes.
        wait(driver).until(
            EC.text_to_be_present_in_element((By.TAG_NAME, "h1"), "RCC")
        )
        wait(driver).until(
            EC.text_to_be_present_in_element(
                (By.CSS_SELECTOR, "p.home-hero__tagline"),
                "Build smarter resumes."
            )
        )

        # Buttons Get started free and Open dashboard are visible.
        wait(driver).until(
            EC.element_to_be_clickable(
                (By.XPATH, "//button[contains(normalize-space(.), 'Get started free')]")
            )
        )

        wait(driver).until(
            EC.element_to_be_clickable(
```

```
                (By.XPATH, "//button[normalize-space()='Open dashboard']")
            )
        )

    finally:
        driver.quit()

if __name__ == "__main__":
    test_home_loads()
```

Test Case 2: Home navigation links scroll to sections

`selenium_tests/logged_out_scripts/test_home_nav_sections.py`

Description

Verifies the anchor links in the public navigation bar. Clicking Features, How it works, Pricing, and FAQ should smooth-scroll the page so the matching section is positioned at the top of the viewport.

Steps

1 Open / with a clean browser state.
2 Click the Features link in the nav bar.
3 Wait until the #features section is within 150px of the current scroll position.
4 Repeat for How it works (#how-it-works), Pricing (#pricing), and FAQ (#faq).

Expected Result

After each click the corresponding section is scrolled into view at the top of the window, allowing for the sticky header offset.

Python Script (Selenium WebDriver)

```
# test_home_nav_sections.py - from /, click Features, How it works, Pricing, FAQ.
# After each click, the matching id (features, how-it-works, pricing, faq) is
# in the viewport (element.location["y"] near execute_script("return window.scrollY")).

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.browser import make_driver, open_fresh, wait


def _near_top(driver, section_id):
    """True once smooth scroll has brought the section near the top of the window."""
    section = driver.find_element(By.ID, section_id)
    scroll_y = driver.execute_script("return window.scrollY")
    # The sticky nav leaves the section about 90px below the scroll position.
    return abs(section.location["y"] - scroll_y) < 150


def test_home_nav_sections():
    driver = make_driver()

    try:
        # Open the home page
        open_fresh(driver, "/")

        # Click Features then find the section and
        # check its vertical position with the page scroll
        wait(driver).until(
            EC.element_to_be_clickable((By.LINK_TEXT, "Features"))
        ).click()

        wait(driver).until(lambda d: _near_top(d, "features"))


        # Click "How it works" then find the section and
        # check its vertical position with the page scroll
        wait(driver).until(
            EC.element_to_be_clickable((By.LINK_TEXT, "How it works"))
        ).click()
```

```
        wait(driver).until(lambda d: _near_top(d, "how-it-works"))


        # Click Pricing then find the section and
        # check its vertical position with the page scroll
        wait(driver).until(
            EC.element_to_be_clickable((By.LINK_TEXT, "Pricing"))
        ).click()

        wait(driver).until(lambda d: _near_top(d, "pricing"))


        # Click FAQ then find the section and
        # check its vertical position with the page scroll
        wait(driver).until(
            EC.element_to_be_clickable((By.LINK_TEXT, "FAQ"))
        ).click()

        wait(driver).until(lambda d: _near_top(d, "faq"))

    finally:
        driver.quit()

if __name__ == "__main__":
    test_home_nav_sections()
```

Test Case 3: FAQ accordion expands

`selenium_tests/logged_out_scripts/test_faq_expand.py`

Description

Tests the FAQ accordion on the home page. Clicking the first question should mark it as expanded and reveal an answer that mentions ATS compatibility.

Steps

1 Open / with a clean browser state.
2 Scroll the #faq section into view so the animated items render.
3 Click the button for How is RCC different from other AI resume tools?
4 Wait until that button has aria-expanded set to true.
5 Read the text of the opened answer panel.

Expected Result

The question button reports aria-expanded="true" and the revealed answer contains the word ATS.

Python Script (Selenium WebDriver)

```
# Click the first FAQ question. aria-expanded becomes true.
# The answer text contains ATS.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.browser import make_driver, open_fresh, wait

QUESTION = "How is RCC different from other AI resume tools?"


def test_faq_expand():
    driver = make_driver()

    try:
        open_fresh(driver, "/")

        # The FAQ starts hidden until it is scrolled into view.
        driver.execute_script(
            "document.getElementById('faq').scrollIntoView({block: 'center'});"
        )

        # The question button also has a chevron, so match the text instead of the whole label.
        button = wait(driver).until(
            EC.element_to_be_clickable(
                (By.XPATH, f"//button[contains(., '{QUESTION}')]")
            )
        )
        button.click()

        wait(driver).until(
            lambda d: d.find_element(
                By.XPATH, f"//button[contains(., '{QUESTION}')]"
            ).get_attribute("aria-expanded") == "true"
        )

        # The answer is clipped while it opens, so read its text content.
        wait(driver).until(
            lambda d: "ATS" in d.find_element(
```

```
                By.CSS_SELECTOR, ".home-faq__item.is-open .home-faq__a"
            ).get_attribute("textContent")
        )

    finally:
        driver.quit()


if __name__ == "__main__":
    test_faq_expand()
```

Test Case 4: Public theme toggle

`selenium_tests/logged_out_scripts/test_theme_public.py`

Description

Checks the light/dark theme switch in the public header. The theme is stored on the data-theme attribute of the html element and should flip when the toggle is clicked.

Steps

1 Open / with a clean browser state.
2 Read the current data-theme attribute from document.documentElement.
3 Click button.theme-toggle in the header.
4 Wait until data-theme differs from the value read in step 2.

Expected Result

data-theme changes from light to dark (or dark to light) after a single click.

Python Script (Selenium WebDriver)

```
# Read data-theme on the page, click the theme button, and check it flipped.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.browser import make_driver, open_fresh, wait


def test_theme_public():
    driver = make_driver()

    try:
        open_fresh(driver, "/")

        before = driver.execute_script(
            "return document.documentElement.getAttribute('data-theme')"
        )

        wait(driver).until(
            EC.element_to_be_clickable((By.CSS_SELECTOR, "button.theme-toggle"))
        ).click()

        wait(driver).until(
            lambda d: d.execute_script(
                "return document.documentElement.getAttribute('data-theme')"
            ) != before
        )

    finally:
        driver.quit()


if __name__ == "__main__":
    test_theme_public()
```

Test Case 5: Nav Sign in link opens login

`selenium_tests/logged_out_scripts/test_nav_sign_in.py`

Description

Verifies the guest call-to-action in the header. Clicking Sign in should route to the login form, which has an email field but no name field.

Steps

1 Open / with a clean browser state.
2 Click the Sign in link in the header.
3 Wait until the URL ends with /login.
4 Wait for #auth-email to be present.
5 Confirm #auth-name is absent.

Expected Result

The browser is on /login, #auth-email exists, and #auth-name does not.

Python Script (Selenium WebDriver)

```
# Click Sign in in the nav. URL should end on /login.
# The login form has the email field and no name field.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.browser import make_driver, open_fresh, wait


def test_nav_sign_in():
    driver = make_driver()

    try:
        open_fresh(driver, "/")

        wait(driver).until(
            EC.element_to_be_clickable((By.LINK_TEXT, "Sign in"))
        ).click()

        wait(driver).until(lambda d: d.current_url.rstrip("/").endswith("/login"))
        wait(driver).until(EC.presence_of_element_located((By.ID, "auth-email")))
        assert driver.find_elements(By.ID, "auth-name") == []

    finally:
        driver.quit()


if __name__ == "__main__":
    test_nav_sign_in()
```

Test Case 6: Hero CTA opens signup

`selenium_tests/logged_out_scripts/test_cta_signup.py`

Description

Tests the hero Get started free button. It should take the visitor to the signup form, which includes a full-name field and a Create Account submit button.

Steps

1 Open / with a clean browser state.
2 Click the hero Get started free button.
3 Wait until the URL ends with /signup.
4 Wait for #auth-name to be present.
5 Read the text of the submit button.

Expected Result

The browser is on /signup, #auth-name exists, and the submit button says Create Account.

Python Script (Selenium WebDriver)

```
# Click the hero Get started free button.
# Should land on /signup with the name field and a Create Account button.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.browser import make_driver, open_fresh, wait


def test_cta_signup():
    driver = make_driver()

    try:
        open_fresh(driver, "/")

        # First match is the hero button. The other one is further down the page.
        wait(driver).until(
            EC.element_to_be_clickable(
                (By.XPATH, "//button[contains(normalize-space(.), 'Get started free')]")
            )
        ).click()

        wait(driver).until(lambda d: d.current_url.rstrip("/").endswith("/signup"))
        wait(driver).until(EC.presence_of_element_located((By.ID, "auth-name")))

        submit = driver.find_element(By.CSS_SELECTOR, "button.auth-submit-btn")
        assert submit.text.strip() == "Create Account"

    finally:
        driver.quit()


if __name__ == "__main__":
    test_cta_signup()
```

Test Case 7: Open dashboard redirects guests

`selenium_tests/logged_out_scripts/test_open_dashboard_logged_out.py`

Description

Checks that the protected dashboard route cannot be reached by a visitor. Clicking Open dashboard while logged out should redirect to the login page.

Steps

1 Open / with a clean browser state.
2 Click the Open dashboard button.
3 Wait until the URL contains /login.

Expected Result

The browser lands on /login instead of /app.

Python Script (Selenium WebDriver)

```
# Click Open dashboard while logged out.
# The protected page should send the browser to /login.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.browser import make_driver, open_fresh, wait, wait_url_contains


def test_open_dashboard_logged_out():
    driver = make_driver()

    try:
        open_fresh(driver, "/")

        wait(driver).until(
            EC.element_to_be_clickable(
                (By.XPATH, "//button[normalize-space()='Open dashboard']")
            )
        ).click()

        wait_url_contains(driver, "/login")

    finally:
        driver.quit()


if __name__ == "__main__":
    test_open_dashboard_logged_out()
```

Test Case 8: Terms page loads

`selenium_tests/logged_out_scripts/test_terms_page.py`

Description

Opens the Terms of Service page directly by URL and checks its heading.

Steps

1 Open /terms with a clean browser state.
2 Wait for the h1 heading to contain Terms of Service.

Expected Result

The page heading reads Terms of Service.

Python Script (Selenium WebDriver)

```
# Open /terms directly. The heading should be Terms of Service.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.browser import make_driver, open_fresh, wait


def test_terms_page():
    driver = make_driver()

    try:
        open_fresh(driver, "/terms")

        wait(driver).until(
            EC.text_to_be_present_in_element((By.TAG_NAME, "h1"), "Terms of Service")
        )

    finally:
        driver.quit()


if __name__ == "__main__":
    test_terms_page()
```

Test Case 9: Footer Terms link

`selenium_tests/logged_out_scripts/test_footer_terms.py`

Description

Verifies the Terms link in the site footer navigates to the same Terms of Service page as the direct URL.

Steps

1 Open / with a clean browser state.
2 Scroll the footer Terms link into view with instant scrolling.
3 Wait until the link is fully inside the viewport, then click it.
4 Wait for the h1 heading to contain Terms of Service.

Expected Result

The browser navigates to /terms and the heading reads Terms of Service.

Python Script (Selenium WebDriver)

```
# From the home page, click the Terms link in the footer.
# Same heading as opening /terms directly.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.browser import make_driver, open_fresh, wait


def test_footer_terms():
    driver = make_driver()

    try:
        open_fresh(driver, "/")

        # Smooth scrolling finishes after the click, so jump there instead.
        driver.execute_script(
            """
            document.documentElement.style.scrollBehavior = 'auto';
            document.querySelector("footer a[href='/terms']").scrollIntoView({block: 'center'});
            """
        )
        wait(driver).until(
            lambda d: d.execute_script(
                """
                const r = document.querySelector("footer a[href='/terms']").getBoundingClientRect();
                return r.top >= 0 && r.bottom <= window.innerHeight;
                """
            )
        )
        wait(driver).until(
            EC.element_to_be_clickable((By.CSS_SELECTOR, "footer a[href='/terms']"))
        ).click()

        wait(driver).until(
            EC.text_to_be_present_in_element((By.TAG_NAME, "h1"), "Terms of Service")
        )

    finally:
        driver.quit()
if __name__ == "__main__":
    test_footer_terms()
```

Test Case 10: Footer Contact popover

`selenium_tests/logged_out_scripts/test_footer_contact.py`

Description

Tests the Contact button in the footer. It should open a small dialog that displays the support email address.

Steps

1 Open / with a clean browser state.
2 Scroll the footer Contact button into view with instant scrolling.
3 Wait until the button is fully inside the viewport, then click it.
4 Wait for the dialog with aria-label Contact email to be visible.
5 Read the dialog text.

Expected Result

A dialog labelled Contact email appears and contains arcc.resume@gmail.com.

Python Script (Selenium WebDriver)

```
# Click Contact in the footer. The dialog should show the contact email.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.browser import make_driver, open_fresh, wait


def test_footer_contact():
    driver = make_driver()

    try:
        open_fresh(driver, "/")

        # Smooth scrolling finishes after the click, so jump there instead.
        driver.execute_script(
            """
            document.documentElement.style.scrollBehavior = 'auto';
            document.querySelector('footer button.site-footer__contact-btn').scrollIntoView({block:
                'center'});
            """
        )
        wait(driver).until(
            lambda d: d.execute_script(
                """
                const r = document.querySelector('footer
                    button.site-footer__contact-btn').getBoundingClientRect();
                return r.top >= 0 && r.bottom <= window.innerHeight;
                """
            )
        )
        wait(driver).until(
            EC.element_to_be_clickable((By.CSS_SELECTOR, "footer button.site-footer__contact-btn"))
        ).click()

        dialog = wait(driver).until(
            EC.visibility_of_element_located((By.CSS_SELECTOR, "[aria-label='Contact email']"))
        )
        assert "arcc.resume@gmail.com" in dialog.text
    finally:
```

```
        driver.quit()


if __name__ == "__main__":
    test_footer_contact()
```

Test Case 11: Signup form fields

`selenium_tests/logged_out_scripts/test_signup_form.py`

Description

Checks the structure of the signup form: three text fields, the free-account subtitle, and the Google sign-up area. The Google button is only checked for presence and is never clicked.

Steps

1 Open /signup with a clean browser state.
2 Wait for #auth-name, #auth-email, and #auth-password to be present.
3 Wait for the subtitle to contain Get started with RCC for free.
4 Wait for the Google button wrapper (.auth-google-wrap) to be present.

Expected Result

All three inputs exist, the subtitle reads Get started with RCC for free, and the Google sign-up wrapper is on the page.

Python Script (Selenium WebDriver)

```
# /signup shows the name, email, and password fields.
# Also the free-account subtitle. Do not click Google.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.browser import make_driver, open_fresh, wait


def test_signup_form():
    driver = make_driver()

    try:
        open_fresh(driver, "/signup")

        wait(driver).until(EC.presence_of_element_located((By.ID, "auth-name")))
        wait(driver).until(EC.presence_of_element_located((By.ID, "auth-email")))
        wait(driver).until(EC.presence_of_element_located((By.ID, "auth-password")))

        wait(driver).until(
            EC.text_to_be_present_in_element(
                (By.CSS_SELECTOR, ".auth-card__sub"),
                "Get started with RCC for free",
            )
        )

        # Google's own button is drawn in this wrapper. Leave it alone.
        wait(driver).until(
            EC.presence_of_element_located((By.CSS_SELECTOR, ".auth-google-wrap"))
        )

    finally:
        driver.quit()


if __name__ == "__main__":
    test_signup_form()
```

Test Case 12: Auth tab switching

`selenium_tests/logged_out_scripts/test_auth_tab_switch.py`

Description

Verifies the Sign In / Sign Up tabs on the auth card. Switching to Sign In should remove the name field and relabel the submit button; switching back should restore it.

Steps

1 Open /signup and wait for #auth-name.
2 Click the Sign In tab.
3 Wait until #auth-name is gone and the submit button reads Sign In.
4 Click the Sign Up tab.
5 Wait for #auth-name to return.

Expected Result

On Sign In the name field is removed and the submit button says Sign In; on Sign Up the name field is back.

Python Script (Selenium WebDriver)

```
# On /signup, switch to Sign In and back to Sign Up.
# The name field should disappear and then come back.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.browser import make_driver, open_fresh, wait


def test_auth_tab_switch():
    driver = make_driver()

    try:
        open_fresh(driver, "/signup")
        wait(driver).until(EC.presence_of_element_located((By.ID, "auth-name")))

        wait(driver).until(
            EC.element_to_be_clickable((By.XPATH, "//button[normalize-space()='Sign In']"))
        ).click()

        wait(driver).until(lambda d: d.find_elements(By.ID, "auth-name") == [])
        # The card fades in, so wait until the label is actually visible.
        wait(driver).until(
            EC.text_to_be_present_in_element(
                (By.CSS_SELECTOR, "button.auth-submit-btn"),
                "Sign In",
            )
        )

        wait(driver).until(
            EC.element_to_be_clickable((By.XPATH, "//button[normalize-space()='Sign Up']"))
        ).click()

        wait(driver).until(EC.presence_of_element_located((By.ID, "auth-name")))

    finally:
        driver.quit()
if __name__ == "__main__":
    test_auth_tab_switch()
```

Test Case 13: Signup rejects a short password

`selenium_tests/logged_out_scripts/test_signup_short_password.py`

Description

Checks the browser-side password rule on signup. The field has minLength 8, so a five-character password must be stopped before the form is submitted to the server.

Steps

1 Open /signup with a clean browser state.
2 Type a name, a unique email, and the password short.
3 Click Create Account.
4 Confirm the Check your email heading and the #auth-code field are absent.
5 Read the validationMessage of #auth-password.

Expected Result

The form stays on the signup card, no verification step appears, and the password field reports a non-empty validation message.

Python Script (Selenium WebDriver)

```
# A password shorter than 8 characters should be stopped by the browser.
# The page stays on the signup card. The code field never appears.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.browser import make_driver, open_fresh, wait


def test_signup_short_password():
    driver = make_driver()

    try:
        open_fresh(driver, "/signup")

        wait(driver).until(EC.presence_of_element_located((By.ID, "auth-name"))).send_keys(
            "RCC Selenium"
        )
        driver.find_element(By.ID, "auth-email").send_keys("short-password-check@example.com")
        driver.find_element(By.ID, "auth-password").send_keys("short")

        wait(driver).until(
            EC.element_to_be_clickable(
                (By.XPATH, "//button[normalize-space()='Create Account']")
            )
        ).click()

        # Still on the signup card.
        assert driver.find_elements(By.CSS_SELECTOR, "h2.auth-verify__title") == []
        assert driver.find_elements(By.ID, "auth-code") == []

        message = driver.find_element(By.ID, "auth-password").get_attribute("validationMessage")
        assert message

    finally:
        driver.quit()
if __name__ == "__main__":
```

```
test_signup_short_password()
```

Test Case 14: Signup rejects an unlisted test address

`selenium_tests/logged_out_scripts/test_signup_rejects_unlisted.py`

Description

Boundary test for the local test-account allow-list. rcc999999@example.test is a well-formed .test address but is not one of the 100 permitted accounts, so the server should reject it instead of starting verification.

Steps

1 Open /signup with a clean browser state.
2 Enter name RCC Selenium, email rcc999999@example.test, password Rcc-test-pass-x.
3 Click Create Account.
4 Wait for an error message with class status-text--error.
5 Confirm the Check your email heading and #auth-code are absent.

Expected Result

An error is shown on the signup card and the verification step is never reached.

Python Script (Selenium WebDriver)

```
# rcc999999@example.test looks like a test address but it is not one of the
# 100 allowed accounts. Signup should show an error and stay off the code screen.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.browser import make_driver, open_fresh, wait


def test_signup_rejects_unlisted():
    driver = make_driver()

    try:
        open_fresh(driver, "/signup")

        wait(driver).until(EC.presence_of_element_located((By.ID, "auth-name"))).send_keys(
            "RCC Selenium"
        )
        driver.find_element(By.ID, "auth-email").send_keys("rcc999999@example.test")
        driver.find_element(By.ID, "auth-password").send_keys("Rcc-test-pass-x")

        wait(driver).until(
            EC.element_to_be_clickable(
                (By.XPATH, "//button[normalize-space()='Create Account']")
            )
        ).click()

        wait(driver).until(
            EC.visibility_of_element_located((By.CSS_SELECTOR, ".status-text--error"))
        )
        assert driver.find_elements(By.CSS_SELECTOR, "h2.auth-verify__title") == []
        assert driver.find_elements(By.ID, "auth-code") == []

    finally:
        driver.quit()


if __name__ == "__main__":
    test_signup_rejects_unlisted()
```

Test Case 15: Login form fields

`selenium_tests/logged_out_scripts/test_login_form.py`

Description

Checks the structure of the login form: the welcome subtitle, the email and password fields, no name field, and the Google sign-in area. The Google button is never clicked.

Steps

1 Open /login with a clean browser state.
2 Wait for the subtitle to contain Welcome back to RCC.
3 Wait for #auth-email and #auth-password to be present.
4 Confirm #auth-name is absent.
5 Wait for the Google button wrapper (.auth-google-wrap) to be present.

Expected Result

The subtitle reads Welcome back to RCC, both credential fields exist, there is no name field, and the Google wrapper is on the page.

Python Script (Selenium WebDriver)

```
# /login shows the welcome subtitle, email, and password.
# No name field. Do not click Google.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.browser import make_driver, open_fresh, wait


def test_login_form():
    driver = make_driver()

    try:
        open_fresh(driver, "/login")

        wait(driver).until(
            EC.text_to_be_present_in_element(
                (By.CSS_SELECTOR, ".auth-card__sub"),
                "Welcome back to RCC",
            )
        )
        wait(driver).until(EC.presence_of_element_located((By.ID, "auth-email")))
        wait(driver).until(EC.presence_of_element_located((By.ID, "auth-password")))
        assert driver.find_elements(By.ID, "auth-name") == []

        # Google's own button is drawn in this wrapper. Leave it alone.
        wait(driver).until(
            EC.presence_of_element_located((By.CSS_SELECTOR, ".auth-google-wrap"))
        )

    finally:
        driver.quit()


if __name__ == "__main__":
    test_login_form()
```

Test Case 16: Login with wrong credentials

`selenium_tests/logged_out_scripts/test_login_bad_password.py`

Description

Verifies the error path for sign-in. Submitting an unknown email with a wrong password should show Invalid credentials and keep the user on the login page.

Steps

1 Open /login with a clean browser state.
2 Type nobody-rcc-test@example.com and wrong-password-1.
3 Click the Sign In submit button.
4 Wait for the .status-text--error message to be visible.
5 Confirm the URL still contains /login.

Expected Result

The error text is Invalid credentials and the browser stays on /login.

Python Script (Selenium WebDriver)

```
# A made-up email and a wrong password should stay on /login
# and show Invalid credentials.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.browser import make_driver, open_fresh, wait, wait_url_contains


def test_login_bad_password():
    driver = make_driver()

    try:
        open_fresh(driver, "/login")

        wait(driver).until(EC.presence_of_element_located((By.ID, "auth-email"))).send_keys(
            "nobody-rcc-test@example.com"
        )
        driver.find_element(By.ID, "auth-password").send_keys("wrong-password-1")

        # The tab is also labelled Sign In, so click the submit button itself.
        wait(driver).until(
            EC.element_to_be_clickable((By.CSS_SELECTOR, "button.auth-submit-btn"))
        ).click()

        error = wait(driver).until(
            EC.visibility_of_element_located((By.CSS_SELECTOR, ".status-text--error"))
        )
        assert error.text.strip() == "Invalid credentials"
        wait_url_contains(driver, "/login")

    finally:
        driver.quit()


if __name__ == "__main__":
    test_login_bad_password()
```

Test Case 17: Protected routes redirect to login

`selenium_tests/logged_out_scripts/test_protected_redirects.py`

Description

Checks every dashboard route while logged out. Each one is wrapped in a protected route and must redirect a guest to the login page.

Steps

1 For each of /app, /app/analyze, /app/interview, /app/history, /app/settings:
2 Open the path with a clean browser state.
3 Wait until the URL contains /login.

Expected Result

All five protected routes end on /login.

Python Script (Selenium WebDriver)

```
# Logged out, every dashboard route should bounce to /login.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from support.browser import make_driver, open_fresh, wait_url_contains

PATHS = [
    "/app",
    "/app/analyze",
    "/app/interview",
    "/app/history",
    "/app/settings",
]


def test_protected_redirects():
    driver = make_driver()

    try:
        for path in PATHS:
            open_fresh(driver, path)
            wait_url_contains(driver, "/login")

    finally:
        driver.quit()


if __name__ == "__main__":
    test_protected_redirects()
```

Test Case 18: Legacy routes redirect to login

`selenium_tests/logged_out_scripts/test_legacy_redirects.py`

Description

Checks the old URLs that were kept as redirects. /verify-email goes straight to login; /upload, /job, /dashboard, /interview, and /verify-pending forward into protected /app routes, which then redirect a guest to login.

Steps

1 For each of /verify-email, /upload, /job, /dashboard, /interview, /verify-pending:
2 Open the path with a clean browser state.
3 Wait until the URL contains /login.

Expected Result

All six legacy routes end on /login.

Python Script (Selenium WebDriver)

```
# Old routes should end up on /login.
# /verify-email goes there directly. The others go into /app first, which then redirects.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from support.browser import make_driver, open_fresh, wait_url_contains

PATHS = [
    "/verify-email",
    "/upload",
    "/job",
    "/dashboard",
    "/interview",
    "/verify-pending",
]


def test_legacy_redirects():
    driver = make_driver()

    try:
        for path in PATHS:
            open_fresh(driver, path)
            wait_url_contains(driver, "/login")

    finally:
        driver.quit()


if __name__ == "__main__":
    test_legacy_redirects()
```

Part 2 - Signup Test Case

Test Case 19: Full signup and email verification

`selenium_tests/full_signup_script/test_signup_full.py`

Description

Completes an end-to-end account creation. The script picks an unused address from the local test pool (rcc000001 to rcc000099 @example.test), submits the signup form, enters the matching six-digit verification code, and lands on the dashboard. No inbox or database is read; the server itself records which addresses are already taken.

Steps

1 Open /signup with a clean browser state.
2 Call claim_signup, which shuffles pool indices 1-99 and submits the form until the server accepts one.
3 Wait for the Check your email heading.
4 Type the six-digit code that matches the accepted address into #auth-code.
5 Click Verify and continue.
6 Wait until the URL contains /app and the h1 contains Welcome.

Expected Result

A new verified account is created and the browser arrives on the dashboard with a Welcome heading. Nothing is written to disk.

Python Script (Selenium WebDriver)

```
import sys
from pathlib import Path

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from support.browser import make_driver, open_fresh, wait, wait_url_contains
from support.pool import claim_signup


def test_signup_full():
    driver = make_driver()

    try:
        # Open a clean signup page
        open_fresh(driver, "/signup")

        # Take the first free account from 1–99
        account = claim_signup(driver)

        # Wait for the code screen
        wait(driver).until(
            EC.text_to_be_present_in_element(
                (By.CSS_SELECTOR, "h2.auth-verify__title"),
                "Check your email",
            )
        )

        # Type that same code and continue
        wait(driver).until(EC.presence_of_element_located((By.ID, "auth-code"))).send_keys(
            account["code"]
        )
```

```
        wait(driver).until(
            EC.element_to_be_clickable(
                (By.XPATH, "//button[normalize-space()='Verify and continue']")
            )
        ).click()

        # Land on the Dashboard
        wait_url_contains(driver, "/app")
        wait(driver).until(
            EC.text_to_be_present_in_element((By.TAG_NAME, "h1"), "Welcome")
        )

    finally:
        driver.quit()

if __name__ == "__main__":
    test_signup_full()
```

Part 3 - Logged-in Test Cases

Test Case 20: Login succeeds

`selenium_tests/logged_in_scripts/test_login_success.py`

Description

Signs in with the permanent test account and confirms the dashboard overview loads.

Steps

1 Sign in as the test account rcc000000@example.test using support/auth.py.
2 Confirm the URL contains /app.
3 Wait for the h1 heading to contain Welcome.

Expected Result

The browser is on /app and the heading contains Welcome.

Python Script (Selenium WebDriver)

```
# Log in as account 0. URL should contain /app and the heading says Welcome.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.auth import login
from support.browser import make_driver, wait


def test_login_success():
    driver = make_driver()

    try:
        login(driver)

        assert "/app" in driver.current_url
        wait(driver).until(
            EC.text_to_be_present_in_element((By.TAG_NAME, "h1"), "Welcome")
        )

    finally:
        driver.quit()


if __name__ == "__main__":
    test_login_success()
```

Test Case 21: Overview page content

`selenium_tests/logged_in_scripts/test_overview_content.py`

Description

Checks the dashboard overview shows its three statistic cards and three quick-action cards.

Steps

1 Sign in as the test account rcc000000@example.test using support/auth.py.
2 Wait for the Welcome heading.
3 Wait for each of Total Analyses, Average Score, Best Score, New Analysis, Practice Interview, and View History to be visible.

Expected Result

All six labels are visible on the overview page.

Python Script (Selenium WebDriver)

```
# The overview page shows the three stat labels and the three quick actions.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.auth import login
from support.browser import make_driver, wait

LABELS = [
    "Total Analyses",
    "Average Score",
    "Best Score",
    "New Analysis",
    "Practice Interview",
    "View History",
]


def test_overview_content():
    driver = make_driver()

    try:
        login(driver)

        wait(driver).until(
            EC.text_to_be_present_in_element((By.TAG_NAME, "h1"), "Welcome")
        )

        # The cards fade in one by one, so wait for each label.
        for label in LABELS:
            wait(driver).until(
                EC.visibility_of_element_located(
                    (By.XPATH, f"//*[normalize-space()='{label}']")
                )
            )

    finally:
        driver.quit()


if __name__ == "__main__":
    test_overview_content()
```

Test Case 22: Sidebar navigation

`selenium_tests/logged_in_scripts/test_sidebar_nav.py`

Description

Clicks each sidebar link in turn and checks that the correct dashboard page loads.

Steps

1 Sign in as the test account rcc000000@example.test using support/auth.py.
2 Click Analyze; wait for /app/analyze and #analyze-jd.
3 Click Interview; wait for /app/interview and #interview-jd.
4 Click History; wait for /app/history and the heading Your activity.
5 Click Settings; wait for /app/settings and the heading Account.
6 Click Overview; wait for the heading Welcome.

Expected Result

Each link routes to its page and the expected element or heading is present.

Python Script (Selenium WebDriver)

```
# Click each sidebar link and check the page that loads.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.auth import login
from support.browser import make_driver, wait, wait_url_contains


def click_sidebar(driver, label):
    wait(driver).until(
        EC.element_to_be_clickable(
            (By.XPATH, f"//aside//a[normalize-space()='{label}']")
        )
    ).click()


def test_sidebar_nav():
    driver = make_driver()

    try:
        login(driver)

        click_sidebar(driver, "Analyze")
        wait_url_contains(driver, "/app/analyze")
        wait(driver).until(EC.presence_of_element_located((By.ID, "analyze-jd")))

        click_sidebar(driver, "Interview")
        wait_url_contains(driver, "/app/interview")
        wait(driver).until(EC.presence_of_element_located((By.ID, "interview-jd")))

        click_sidebar(driver, "History")
        wait_url_contains(driver, "/app/history")
        wait(driver).until(
            EC.text_to_be_present_in_element((By.TAG_NAME, "h1"), "Your activity")
        )

        click_sidebar(driver, "Settings")
        wait_url_contains(driver, "/app/settings")
```

```
        wait(driver).until(
            EC.text_to_be_present_in_element((By.TAG_NAME, "h1"), "Account")
        )

        click_sidebar(driver, "Overview")
        wait(driver).until(
            EC.text_to_be_present_in_element((By.TAG_NAME, "h1"), "Welcome")
        )

    finally:
        driver.quit()


if __name__ == "__main__":
    test_sidebar_nav()
```

Test Case 23: New analysis button

`selenium_tests/logged_in_scripts/test_new_analysis_button.py`

Description

Checks the New analysis shortcut in the sidebar opens the analyze page.

Steps

1 Sign in as the test account rcc000000@example.test using support/auth.py.
2 Click the New analysis button in the sidebar.
3 Wait until the URL contains /app/analyze.

Expected Result

The browser navigates to /app/analyze.

Python Script (Selenium WebDriver)

```
# The New analysis button in the sidebar goes to /app/analyze.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.auth import login
from support.browser import make_driver, wait, wait_url_contains


def test_new_analysis_button():
    driver = make_driver()

    try:
        login(driver)

        wait(driver).until(
            EC.element_to_be_clickable((By.CSS_SELECTOR, "button.dash-sidebar__new"))
        ).click()

        wait_url_contains(driver, "/app/analyze")

    finally:
        driver.quit()


if __name__ == "__main__":
    test_new_analysis_button()
```

Test Case 24: Analyze page fields

`selenium_tests/logged_in_scripts/test_analyze_page.py`

Description

Verifies the analyze page renders its inputs and that the Analyze Resume button is disabled until a resume file has been chosen.

Steps

1 Sign in as the test account rcc000000@example.test using support/auth.py.
2 Navigate to /app/analyze.
3 Wait for #analyze-resume, #analyze-title, and #analyze-jd to be present.
4 Read the disabled attribute of the Analyze Resume button.

Expected Result

All three inputs exist and Analyze Resume carries the disabled attribute.

Python Script (Selenium WebDriver)

```
# The analyze page has the file input, title, and description.
# Analyze Resume stays disabled until a file is chosen.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.auth import login
from support.browser import BASE, make_driver, wait


def test_analyze_page():
    driver = make_driver()

    try:
        login(driver)
        driver.get(BASE + "/app/analyze")

        wait(driver).until(EC.presence_of_element_located((By.ID, "analyze-resume")))
        wait(driver).until(EC.presence_of_element_located((By.ID, "analyze-title")))
        wait(driver).until(EC.presence_of_element_located((By.ID, "analyze-jd")))

        button = wait(driver).until(
            EC.presence_of_element_located(
                (By.XPATH, "//button[contains(normalize-space(.), 'Analyze Resume')]")
            )
        )
        assert button.get_attribute("disabled") is not None

    finally:
        driver.quit()


if __name__ == "__main__":
    test_analyze_page()
```

Test Case 25: Attach a resume file

`selenium_tests/logged_in_scripts/test_analyze_attach_file.py`

Description

Uploads the fixture PDF through the hidden file input and checks the dropzone shows the filename and the Analyze Resume button becomes enabled. The button is not clicked.

Steps

1 Sign in as the test account rcc000000@example.test using support/auth.py.
2 Navigate to /app/analyze.
3 Send the absolute path of Muhammad Zain Resume - Autodesk.pdf to #analyze-resume.
4 Wait for the dropzone to show the filename.
5 Wait for Analyze Resume to be clickable.

Expected Result

The dropzone displays the PDF filename and Analyze Resume is enabled.

Python Script (Selenium WebDriver)

```
# Attach the resume PDF. The dropzone shows the filename and
# Analyze Resume becomes enabled. Do not click it.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.auth import login
from support.browser import BASE, make_driver, wait

RESUME = Path(__file__).resolve().parents[1] / "Muhammad Zain Resume - Autodesk.pdf"


def test_analyze_attach_file():
    driver = make_driver()

    try:
        login(driver)
        driver.get(BASE + "/app/analyze")

        # The file input is invisible but send_keys still works on it.
        wait(driver).until(
            EC.presence_of_element_located((By.ID, "analyze-resume"))
        ).send_keys(str(RESUME))

        wait(driver).until(
            EC.text_to_be_present_in_element(
                (By.CSS_SELECTOR, ".dropzone__file"), RESUME.name
            )
        )

        wait(driver).until(
            EC.element_to_be_clickable(
                (By.XPATH, "//button[contains(normalize-space(.), 'Analyze Resume')]")
            )
        )

    finally:
        driver.quit()
if __name__ == "__main__":
```

```
test_analyze_attach_file()
```

Test Case 26: Run a full analysis

`selenium_tests/logged_in_scripts/test_analyze_run.py`

Description

Performs a real resume analysis against the Autodesk job description. The job description is inserted with a native value setter because typing it would be too slow. This is the only script that spends an LLM call, so it waits up to two minutes.

Steps

1 Sign in as the test account rcc000000@example.test using support/auth.py.
2 Navigate to /app/analyze and upload Muhammad Zain Resume - Autodesk.pdf.
3 Set #analyze-title to MCP/AI Developer.
4 Set #analyze-jd to the contents of Job Description - Autodesk.txt.
5 Click Analyze Resume.
6 Wait up to 120 seconds for .score-ring__num to show a number.
7 Wait for the label Overall match against this job description.

Expected Result

A numeric match score is rendered and the Overall match label appears. No specific score value is asserted.

Python Script (Selenium WebDriver)

```
# Run a real analysis. Upload the resume, fill the job title, paste the
# Autodesk job description, and click Analyze Resume.
# This is the only script that calls the LLM, so it needs LLM_API_KEY in backend/.env.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.auth import login
from support.browser import BASE, make_driver, wait
from support.form import set_react_value

ROOT = Path(__file__).resolve().parents[1]
RESUME = ROOT / "Muhammad Zain Resume - Autodesk.pdf"
JOB_DESCRIPTION = ROOT / "Job Description - Autodesk.txt"


def test_analyze_run():
    driver = make_driver()

    try:
        login(driver)
        driver.get(BASE + "/app/analyze")

        wait(driver).until(
            EC.presence_of_element_located((By.ID, "analyze-resume"))
        ).send_keys(str(RESUME))
        wait(driver).until(
            EC.text_to_be_present_in_element(
                (By.CSS_SELECTOR, ".dropzone__file"), RESUME.name
            )
        )

        set_react_value(driver, "analyze-title", "MCP/AI Developer")
```

```
        # Typing the whole description is too slow, so set it directly.
        set_react_value(
            driver, "analyze-jd", JOB_DESCRIPTION.read_text(encoding="utf-8")
        )

        wait(driver).until(
            EC.element_to_be_clickable(
                (By.XPATH, "//button[contains(normalize-space(.), 'Analyze Resume')]")
            )
        ).click()

        # The app is local but the Gemini call is not. Give it up to two minutes.
        score = wait(driver, 120).until(
            EC.visibility_of_element_located((By.CSS_SELECTOR, ".score-ring__num"))
        )
        wait(driver, 120).until(lambda d: score.text.strip().isdigit())

        wait(driver).until(
            EC.text_to_be_present_in_element(
                (By.CSS_SELECTOR, ".score-hero__label"),
                "Overall match against this job description",
            )
        )

    finally:
        driver.quit()


if __name__ == "__main__":
    test_analyze_run()
```

Test Case 27: Interview page fields

`selenium_tests/logged_in_scripts/test_interview_page.py`

Description

Checks the interview setup page shows the coach heading and its three inputs, and that Begin interview is disabled while the job description is empty.

Steps

1 Sign in as the test account rcc000000@example.test using support/auth.py.
2 Navigate to /app/interview.
3 Wait for the heading RCC Interview Coach.
4 Wait for #interview-role, #interview-jd, and #interview-context to be present.
5 Read the disabled attribute of Begin interview.

Expected Result

The heading and all three inputs exist and Begin interview is disabled.

Python Script (Selenium WebDriver)

```
# The interview page shows the coach heading and all three inputs.
# Begin interview is disabled until there is a job description.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.auth import login
from support.browser import BASE, make_driver, wait


def test_interview_page():
    driver = make_driver()

    try:
        login(driver)
        driver.get(BASE + "/app/interview")

        wait(driver).until(
            EC.text_to_be_present_in_element((By.TAG_NAME, "h2"), "RCC Interview Coach")
        )
        wait(driver).until(EC.presence_of_element_located((By.ID, "interview-role")))
        wait(driver).until(EC.presence_of_element_located((By.ID, "interview-jd")))
        wait(driver).until(EC.presence_of_element_located((By.ID, "interview-context")))

        button = driver.find_element(
            By.XPATH, "//button[contains(normalize-space(.), 'Begin interview')]"
        )
        assert button.get_attribute("disabled") is not None

    finally:
        driver.quit()


if __name__ == "__main__":
    test_interview_page()
```

Test Case 28: Job description enables Begin interview

`selenium_tests/logged_in_scripts/test_interview_jd_enables.py`

Description

Verifies that entering a job description enables the Begin interview button. The button is not clicked so no interview session is started.

Steps

1 Sign in as the test account rcc000000@example.test using support/auth.py.
2 Navigate to /app/interview.
3 Type Software engineer role. Ask about React. into #interview-jd.
4 Wait for Begin interview to be clickable.

Expected Result

Begin interview becomes enabled after the text is entered.

Python Script (Selenium WebDriver)

```
# Typing a job description enables Begin interview. Do not click it.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.auth import login
from support.browser import BASE, make_driver, wait


def test_interview_jd_enables():
    driver = make_driver()

    try:
        login(driver)
        driver.get(BASE + "/app/interview")

        wait(driver).until(
            EC.presence_of_element_located((By.ID, "interview-jd"))
        ).send_keys("Software engineer role. Ask about React.")

        wait(driver).until(
            EC.element_to_be_clickable(
                (By.XPATH, "//button[contains(normalize-space(.), 'Begin interview')]")
            )
        )

    finally:
        driver.quit()


if __name__ == "__main__":
    test_interview_jd_enables()
```

Test Case 29: History page

`selenium_tests/logged_in_scripts/test_history_page.py`

Description

Checks the history page heading and then handles both possible states. If no analyses exist the empty message is the pass. If a clickable card exists it is opened and the detail page must show a score.

Steps

1 Sign in as the test account rcc000000@example.test using support/auth.py.
2 Navigate to /app/history and wait for the heading Your activity.
3 Wait until either Nothing here yet or at least one .history-card is present.
4 If a .history-card--clickable exists, click the first one.
5 Wait until the URL matches /app/history/<id> and .score-ring__num is visible.

Expected Result

Either the empty state is shown, or the first card opens a detail page with a score ring.

Python Script (Selenium WebDriver)

```
# History shows Your activity. Then either the empty state or at least one card.
# If a card can be opened, open it and check the detail page.
# Which branch runs depends on whether test_analyze_run.py has run already.

import re
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.auth import login
from support.browser import BASE, make_driver, wait


def test_history_page():
    driver = make_driver()

    try:
        login(driver)
        driver.get(BASE + "/app/history")

        wait(driver).until(
            EC.text_to_be_present_in_element((By.TAG_NAME, "h1"), "Your activity")
        )

        # Wait until the list has finished loading one way or the other.
        wait(driver).until(
            lambda d: d.find_elements(By.XPATH, "//h2[normalize-space()='Nothing here yet']")
            or d.find_elements(By.CSS_SELECTOR, ".history-card")
        )

        cards = driver.find_elements(By.CSS_SELECTOR, ".history-card--clickable")
        if not cards:
            # Empty state is a valid pass.
            return

        cards[0].click()

        wait(driver).until(lambda d: re.search(r"/app/history/\d+", d.current_url))
        wait(driver).until(
            EC.visibility_of_element_located((By.CSS_SELECTOR, ".score-ring__num"))
```

```
        )

    finally:
        driver.quit()


if __name__ == "__main__":
    test_history_page()
```

Test Case 30: Settings profile

`selenium_tests/logged_in_scripts/test_settings_profile.py`

Description

Checks the settings page heading and that the profile block shows the signed-in email.

Steps

1 Sign in as the test account rcc000000@example.test using support/auth.py.
2 Navigate to /app/settings.
3 Wait for the heading Account.
4 Wait for the profile block to contain rcc000000@example.test.

Expected Result

The heading reads Account and the profile shows rcc000000@example.test.

Python Script (Selenium WebDriver)

```
# Settings shows the Account heading and the logged-in email.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.auth import login
from support.browser import BASE, make_driver, wait
from support.pool import credentials


def test_settings_profile():
    driver = make_driver()

    try:
        login(driver)
        driver.get(BASE + "/app/settings")

        wait(driver).until(
            EC.text_to_be_present_in_element((By.TAG_NAME, "h1"), "Account")
        )
        wait(driver).until(
            EC.text_to_be_present_in_element(
                (By.CSS_SELECTOR, ".settings-profile"),
                credentials(0)["email"],
            )
        )

    finally:
        driver.quit()


if __name__ == "__main__":
    test_settings_profile()
```

Test Case 31: Delete account dialog

`selenium_tests/logged_in_scripts/test_settings_delete_modal.py`

Description

Exercises the delete-account confirmation dialog without deleting anything. The danger button must stay disabled until the word DELETE is typed, and Cancel must close the dialog.

Steps

1 Sign in as the test account rcc000000@example.test using support/auth.py.
2 Navigate to /app/settings and scroll to the Delete account button, then click it.
3 Wait for the dialog title Delete your account?.
4 Confirm the Delete my account button is disabled.
5 Type DELETE into #delete-confirm-input and wait for the button to become enabled.
6 Click Cancel and wait for the dialog to disappear.

Expected Result

The dialog opens, the danger button is disabled until DELETE is typed, and Cancel closes the dialog. The account is never deleted.

Python Script (Selenium WebDriver)

```
# Open the delete dialog, type DELETE so the button enables, then cancel.
# Never confirm. Account 0 has to stay.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.auth import login
from support.browser import BASE, make_driver, wait

DANGER_BUTTON = (By.CSS_SELECTOR, "button.modal__btn--danger")


def test_settings_delete_modal():
    driver = make_driver()

    try:
        login(driver)
        driver.get(BASE + "/app/settings")

        # The danger card is lower on the page and fades in on scroll.
        delete_button = wait(driver).until(
            EC.presence_of_element_located(
                (By.XPATH, "//button[normalize-space()='Delete account']")
            )
        )
        driver.execute_script("arguments[0].scrollIntoView({block: 'center'});", delete_button)
        wait(driver).until(EC.element_to_be_clickable(delete_button)).click()

        wait(driver).until(
            EC.text_to_be_present_in_element(
                (By.ID, "delete-account-title"), "Delete your account?"
            )
        )

        danger = driver.find_element(*DANGER_BUTTON)
        assert danger.get_attribute("disabled") is not None
```

```
        driver.find_element(By.ID, "delete-confirm-input").send_keys("DELETE")
        wait(driver).until(EC.element_to_be_clickable(DANGER_BUTTON))

        driver.find_element(By.XPATH, "//button[normalize-space()='Cancel']").click()
        wait(driver).until(
            EC.invisibility_of_element_located((By.ID, "delete-account-title"))
        )

    finally:
        driver.quit()


if __name__ == "__main__":
    test_settings_delete_modal()
```

Test Case 32: Dashboard theme toggle

`selenium_tests/logged_in_scripts/test_dashboard_theme.py`

Description

Checks the theme switch inside the dashboard sidebar flips data-theme, then clicks again to restore the original value.

Steps

1 Sign in as the test account rcc000000@example.test using support/auth.py.
2 Read the current data-theme attribute.
3 Click .dash-sidebar button.theme-toggle and wait for data-theme to change.
4 Click it again and wait for data-theme to return to the original value.

Expected Result

data-theme flips on the first click and is restored on the second.

Python Script (Selenium WebDriver)

```
# The theme toggle in the sidebar flips data-theme. Click again to put it back.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.auth import login
from support.browser import make_driver, wait

TOGGLE = (By.CSS_SELECTOR, ".dash-sidebar button.theme-toggle")


def current_theme(driver):
    return driver.execute_script(
        "return document.documentElement.getAttribute('data-theme')"
    )


def test_dashboard_theme():
    driver = make_driver()

    try:
        login(driver)

        before = current_theme(driver)

        wait(driver).until(EC.element_to_be_clickable(TOGGLE)).click()
        wait(driver).until(lambda d: current_theme(d) != before)

        wait(driver).until(EC.element_to_be_clickable(TOGGLE)).click()
        wait(driver).until(lambda d: current_theme(d) == before)

    finally:
        driver.quit()


if __name__ == "__main__":
    test_dashboard_theme()
```

Test Case 33: Sidebar collapse and expand

`selenium_tests/logged_in_scripts/test_sidebar_collapse.py`

Description

Verifies the sidebar collapse control. Its aria-label should read Collapse sidebar, switch to Expand sidebar after a click, and switch back after a second click.

Steps

1 Sign in as the test account rcc000000@example.test using support/auth.py.
2 Confirm the collapse button's aria-label is Collapse sidebar, then click it.
3 Wait until the aria-label becomes Expand sidebar.
4 Click it again and wait until the aria-label is Collapse sidebar.

Expected Result

The aria-label toggles between Collapse sidebar and Expand sidebar.

Python Script (Selenium WebDriver)

```
# Collapse the sidebar and expand it again using the same button.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.auth import login
from support.browser import make_driver, wait

COLLAPSE = (By.CSS_SELECTOR, "button.dash-sidebar__collapse-btn")


def test_sidebar_collapse():
    driver = make_driver()

    try:
        login(driver)

        button = wait(driver).until(EC.element_to_be_clickable(COLLAPSE))
        assert button.get_attribute("aria-label") == "Collapse sidebar"
        button.click()

        wait(driver).until(
            lambda d: d.find_element(*COLLAPSE).get_attribute("aria-label")
            == "Expand sidebar"
        )

        driver.find_element(*COLLAPSE).click()
        wait(driver).until(
            lambda d: d.find_element(*COLLAPSE).get_attribute("aria-label")
            == "Collapse sidebar"
        )

    finally:
        driver.quit()


if __name__ == "__main__":
    test_sidebar_collapse()
```

Test Case 34: Logout

`selenium_tests/logged_in_scripts/test_logout.py`

Description

Signs in, logs out through the sidebar, and confirms the dashboard is protected again.

Steps

1 Sign in as the test account rcc000000@example.test using support/auth.py.
2 Click Log out in the sidebar and wait for the URL to contain /login.
3 Navigate to /app again.
4 Wait for the URL to contain /login.

Expected Result

Both after logout and on revisiting /app, the browser is on /login.

Python Script (Selenium WebDriver)

```
# Log in, click Log out, then try /app again. Both times it should be /login.

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from support.auth import login
from support.browser import BASE, make_driver, wait, wait_url_contains


def test_logout():
    driver = make_driver()

    try:
        login(driver)

        wait(driver).until(
            EC.element_to_be_clickable((By.CSS_SELECTOR, "button.dash-sidebar__logout"))
        ).click()
        wait_url_contains(driver, "/login")

        driver.get(BASE + "/app")
        wait_url_contains(driver, "/login")

    finally:
        driver.quit()


if __name__ == "__main__":
    test_logout()
```

References

[1] Grok 4.7 (large language model), accessed through the Cursor IDE, 2026. Used as an assistant during this assignment. The division of the test scripts into logged-out, signup, and logged-in groups was our own; the model was first prompted to turn that outline into a full build plan, which we then refined over several iterations. The test scripts were written from that build plan and this report was produced from the scripts. The model's role was limited to drafting the build plan, formatting, structuring, and cleaning up the code, and formatting the layout of this PDF report. Available: https://cursor.com

[2] Stack Overflow. Community questions and answers consulted for Selenium WebDriver usage in Python, including explicit waits, XPath and CSS selectors, file-input uploads, and handling click interception. Available: https://stackoverflow.com

[3] "Selenium WebDriver Tutorial #1 - Introduction to Selenium," YouTube video. Available: https://www.youtube.com/watch?v=25U-KUpWd1c&list=PLL34mf651faPB-LyEP0-a7Avp_RHO0Lsm